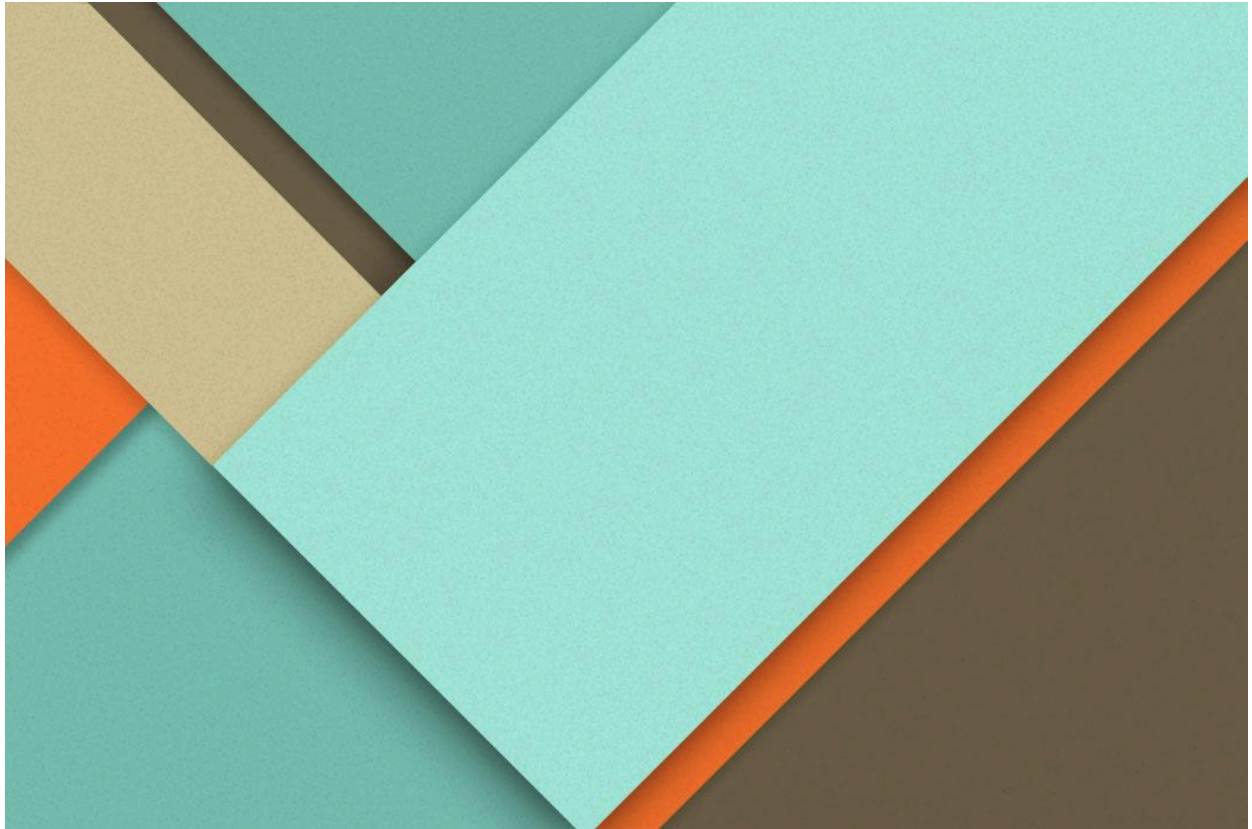




Cine V4: Demo Endpoints

13/05/2026

José Manuel Sánchez Rosal

IES Antonio Gala

14700 Palma del Río (Córdoba)

2 DAM

Índice

Índice.....	1
1. Introducción.....	1
Prueba 1: Acceso a Endpoint Público.....	2
Prueba 2: Intento de Acción Protegida sin Autenticación.....	2
Prueba 3: Registro de un Nuevo Usuario.....	3
Prueba 4: Bloqueo por Rol (RBAC).....	4
Prueba 5: Compra de Entradas (Operación del Cliente).....	5
Prueba 6: Privacidad de Ventas (Validación de Propiedad SpEL).....	5
Prueba 7: Renovación Automática de Sesión (Refresh Token).....	7
Prueba 8: Auditoría Automática JPA (Prueba de Base de Datos).....	8

1. Introducción

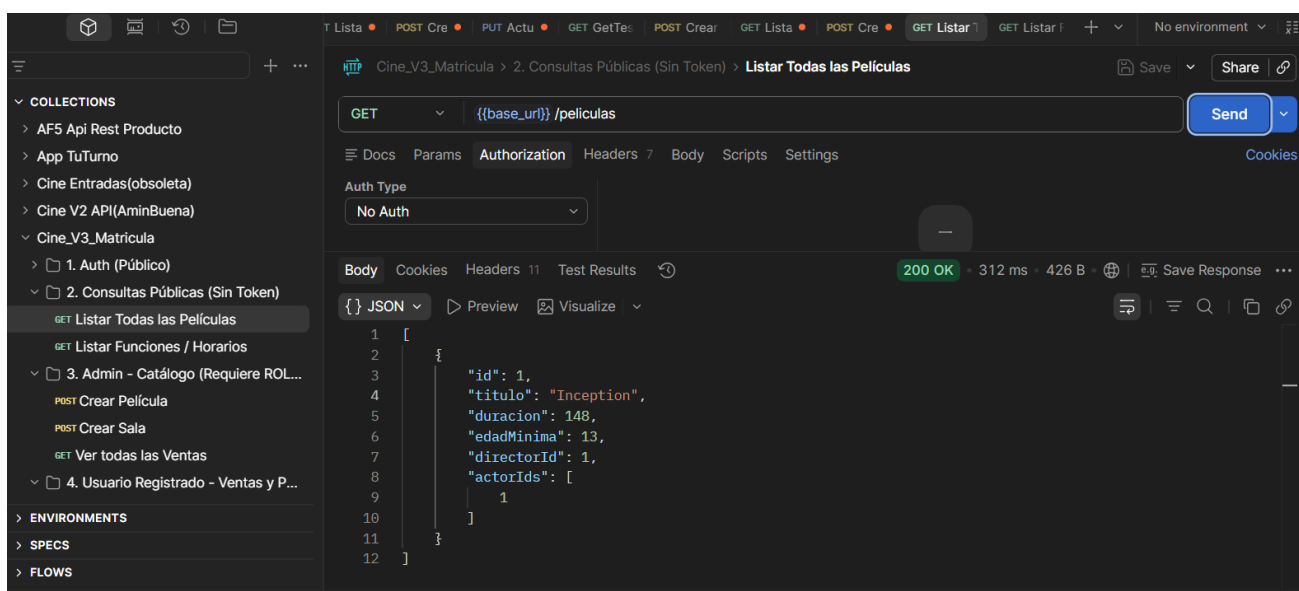
Esta guía está diseñada paso a paso para ejecutar las pruebas en Postman, documentar por qué se hacen, explicar a nivel técnico lo que demuestran y su correspondiente captura.

Prueba 1: Acceso a Endpoint Público

Endpoint a ejecutar: GET /api/v1/peliculas (Carpeta: 2. Consultas Públicas)

Por qué lo probamos: Necesitamos asegurar que cualquier visitante del cine pueda consultar la cartelera sin necesidad de estar registrado o iniciar sesión.

Qué hace y demuestra: Demuestra que la configuración del SecurityFilterChain en nuestra clase SecurityConfig está aplicando correctamente la directiva permitAll() a las rutas seleccionadas, saltándose el chequeo del filtro JWT:



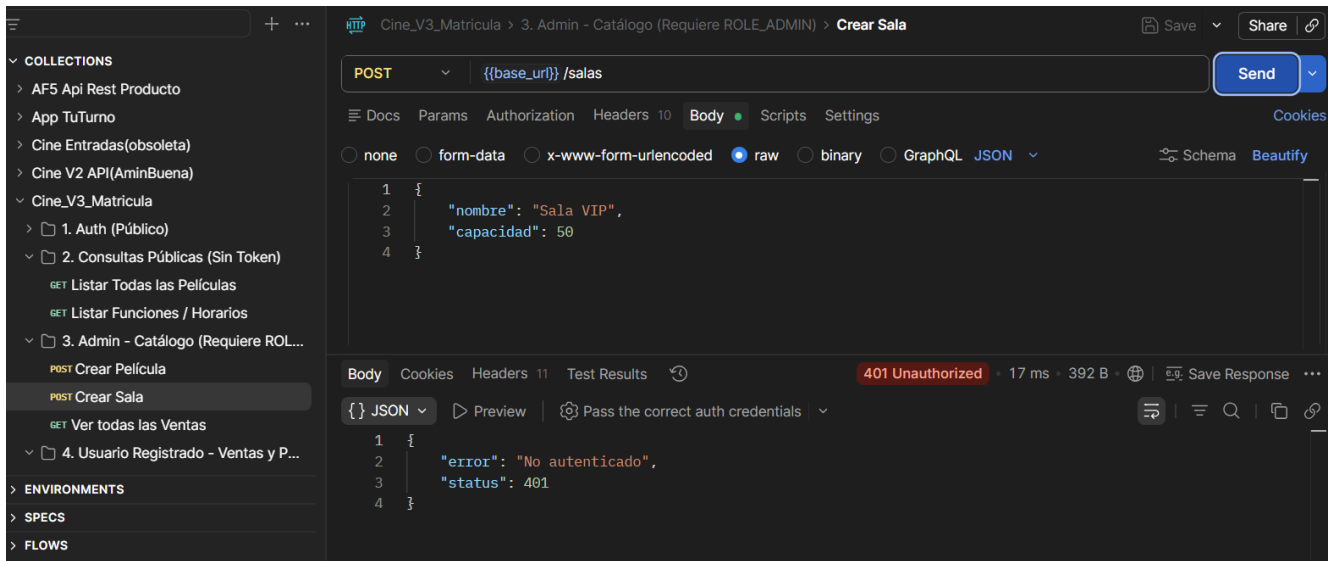
Prueba 2: Intento de Acción Protegida sin Autenticación

Endpoint a ejecutar: POST /api/v1/peliculas (Carpeta: 3. Admin - Catálogo)

Por qué lo probamos: Para comprobar que el sistema bloquea cualquier intento de realizar modificaciones en el catálogo si la API no sabe quién es el emisor de la petición (sin Token).

Qué hace y demuestra: Al no enviar un JWT, la petición es interceptada por nuestro manejador personalizado CustomAuthenticationEntryPoint. Demuestra que hemos eliminado las clásicas respuestas de error HTML de Spring Boot y las hemos sustituido por respuestas JSON puras y profesionales (`{"error": "No autenticado", "status": 401}`).

Captura correspondiente:



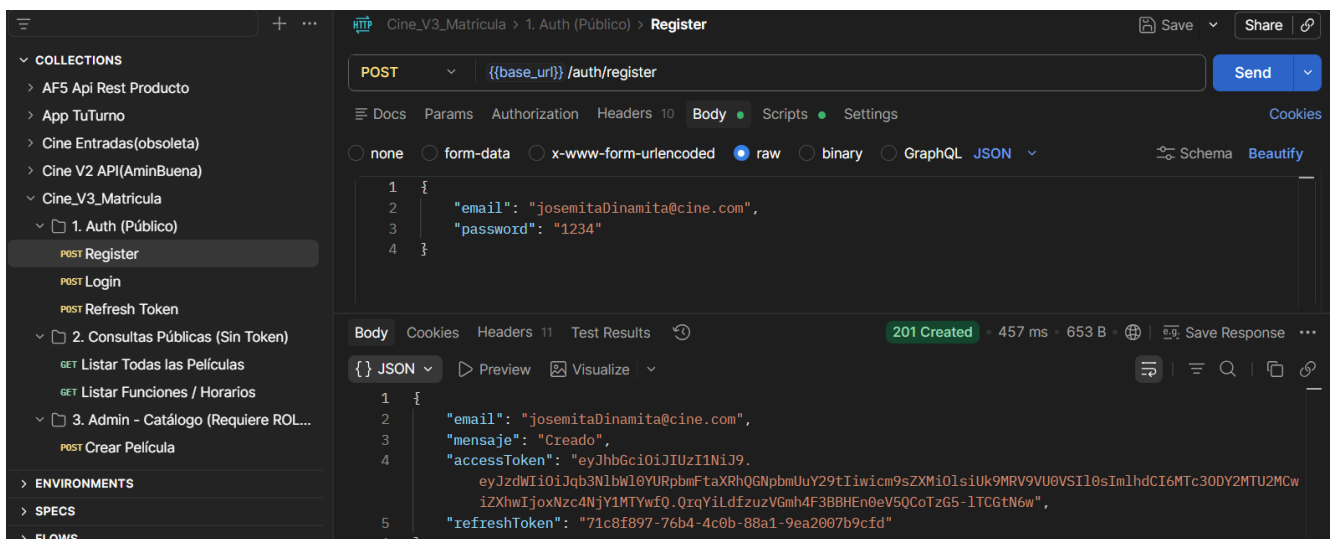
Prueba 3: Registro de un Nuevo Usuario

Endpoint a ejecutar: POST /api/v1/auth/register (Carpeta: 1. Auth)

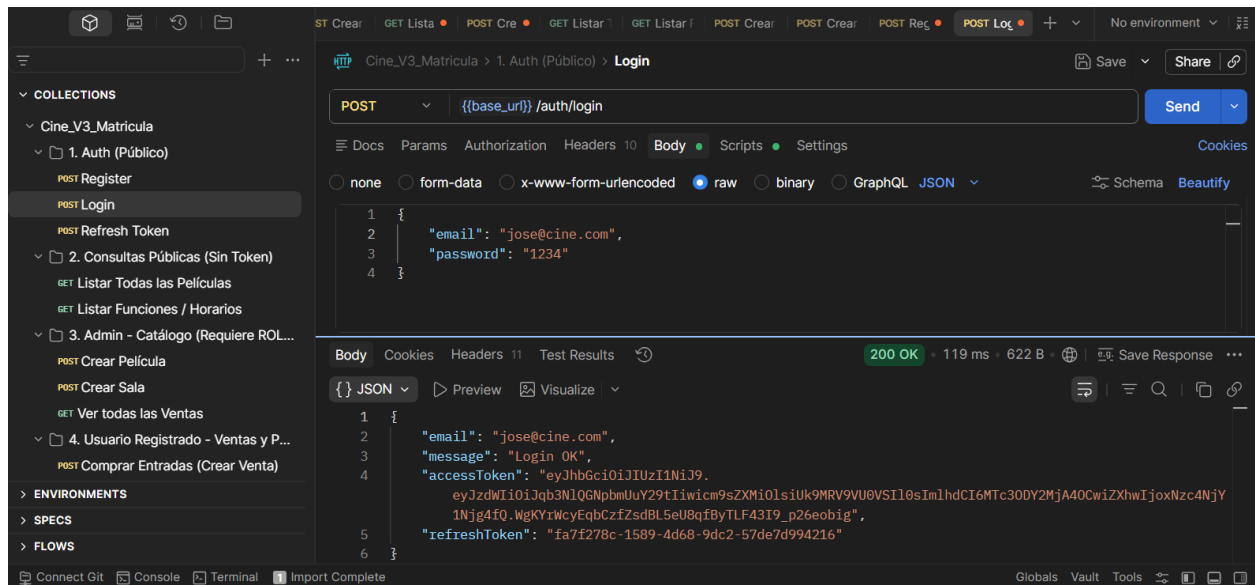
Por qué lo probamos: Para simular la creación de un cliente nuevo y obtener sus credenciales de acceso para las siguientes pruebas.

Qué hace y demuestra: El endpoint encripta la contraseña, le asigna obligatoriamente el rol ROLE_USER, y el servicio UsuarioService genera dos cadenas criptográficas: el JWT (Access Token) y el Refresh Token. Además, gracias al script de Postman en la pestaña "Tests", estos tokens se guardan solos en el entorno de Postman:

1. Registro:



2. Login:

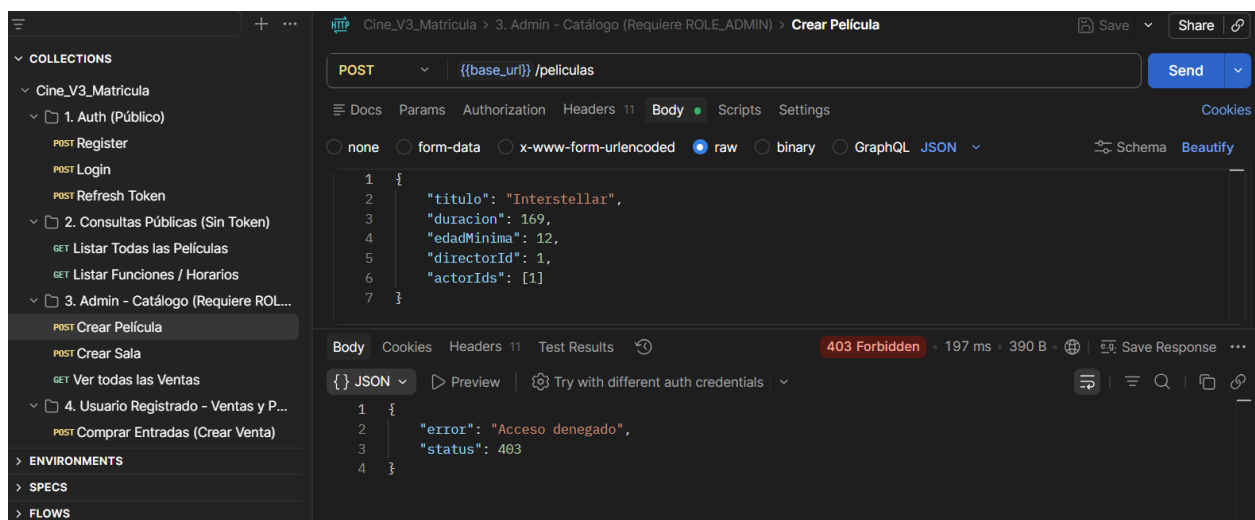


Prueba 4: Bloqueo por Rol (RBAC)

Endpoint a ejecutar: POST /api/v1/peliculas (Carpeta: 3. Admin - Catálogo)

Por qué lo probamos: El usuario se acaba de registrar, por lo que es un cliente (ROLE_USER), pero esta ruta de creación es solo para el administrador (ROLE_ADMIN).

Qué hace y demuestra: Demuestra que la anotación `@PreAuthorize("hasRole('ADMIN')")` puesta en los controladores funciona a la perfección. El filtro de Spring Security lee el token del nuevo usuario, detecta la falta de privilegios, y nuestro CustomAccessDeniedHandler interviene para emitir un error 403 en JSON formateado:

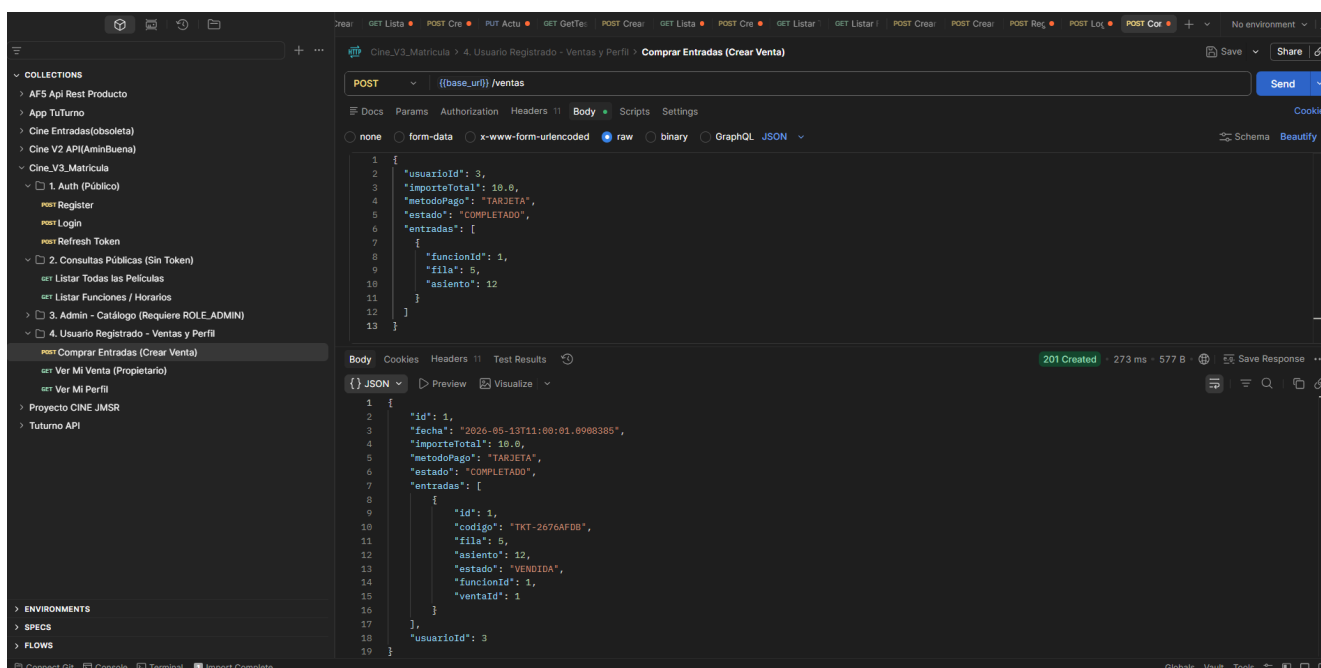


Prueba 5: Compra de Entradas (Operación del Cliente)

Endpoint a ejecutar: POST /api/v1/ventas (Carpeta: 4. Usuario Registrado)

Por qué lo probamos: El cliente necesita poder ejecutar sus propias acciones de negocio (comprar tickets).

Qué hace y demuestra: Demuestra la protección básica (@PreAuthorize("isAuthenticated()")). También evidencia una lógica de seguridad del Backend: el cliente no envía su ID de usuario en la petición. El servidor extrae su email directamente del Token (leyendo el SecurityContextHolder) y lo vincula a la venta automáticamente, previniendo fraudes o suplantaciones de identidad:



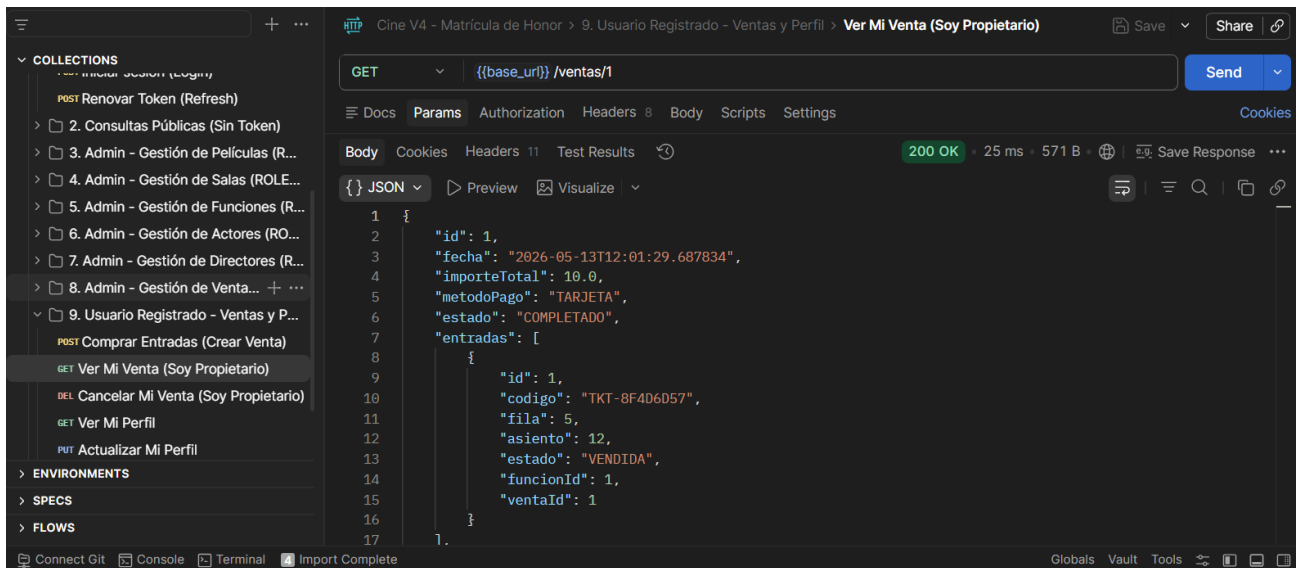
Prueba 6: Privacidad de Ventas (Validación de Propiedad SpEL)

Endpoint a ejecutar: GET /api/v1/ventas/1 (Sustituye el "1" por el ID de la venta creada en el paso 5. Carpeta: 4. Usuario Registrado)

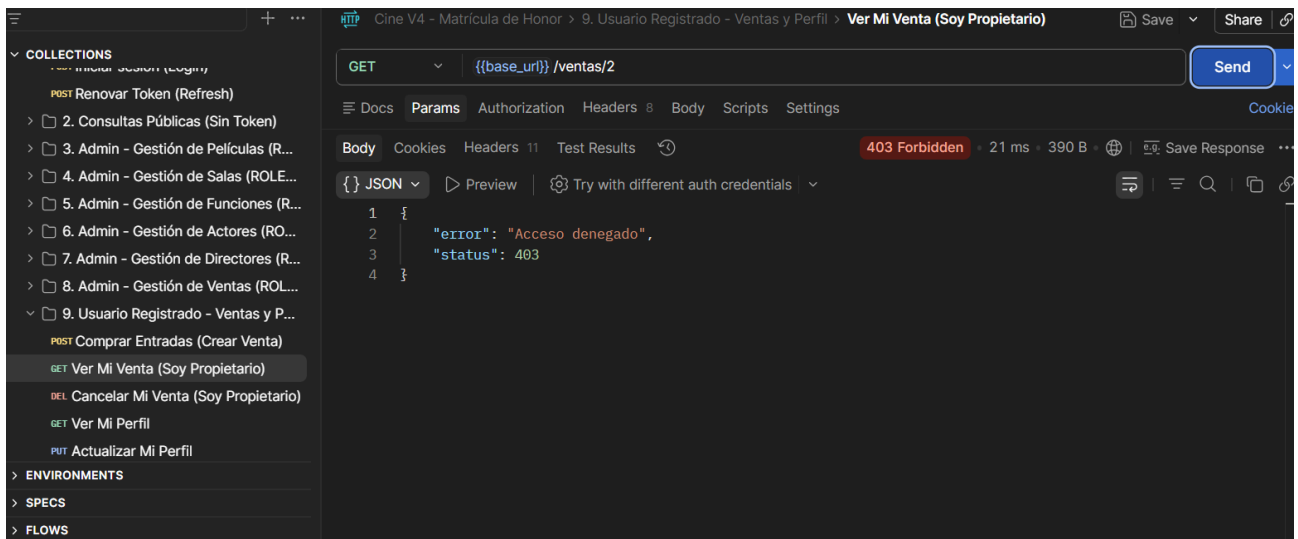
Por qué lo probamos: Para asegurar que un cliente solo pueda ver los detalles de sus propios tickets, no los de otros.

Qué hace y demuestra: Demuestra la avanzada funcionalidad de Spring Expression Language (SpEL). Al consultar, se ejecuta la regla: `@PreAuthorize("hasRole('ADMIN') or @ventaService.esDelUsuario(#id, authentication.principal.username)")`. Esto cruza en tiempo real el dueño de la venta en la Base de Datos con el email del JWT que solicita verla.

1. ID usuario=3, ID venta=1 (autorizado):



2. Intentar ver la venta con ID=2 (Acceso denegado):



NOTA: Nos hemos registrado con otro usuario y realizado otra compra (id=2) para probarlo.

Prueba 7: Renovación Automática de Sesión (Refresh Token)

Endpoint a ejecutar: POST /api/v1/auth/refresh (Carpeta: 1. Auth)

Por qué lo probamos: Para simular que la sesión de 1 hora del accessToken ha caducado, pero el usuario puede conseguir uno nuevo transparente sin volver a introducir su contraseña.

Qué hace y demuestra: Envía el UUID del Refresh Token hacia la API. RefreshTokenService busca este UUID en la Base de Datos, verifica su fecha de caducidad (7 días), e inyecta un JWT recién fabricado si todo es válido, lo que demuestra un flujo de autenticación "State of the Art":

The screenshot shows a REST client interface with a sidebar on the left containing a collection tree. The main panel displays a POST request to `POST {{base_url}} /auth/refresh`. The request body is a JSON object: `{ "refreshToken": "{{refresh_token}}" }`. The response is a 200 OK status with a JSON body: `{ "email": "jose@cine.com", "message": "Token refrescado con éxito.", "accessToken": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJqb3NlOGNpbmUyZ2tIiwicm9sZXMiOiJ1c2Vybm9udGVzIiwiaWF0IjoxNjU2fQ.-v2QPJ60ICkCAGhr4htU0truk76xbgFmGUT3hN1344", "refreshToken": "8af913ce-6163-4b34-8928-b5133f8a48e0" }`.

```
1 {
2   "refreshToken": "{{refresh_token}}"
3 }
```

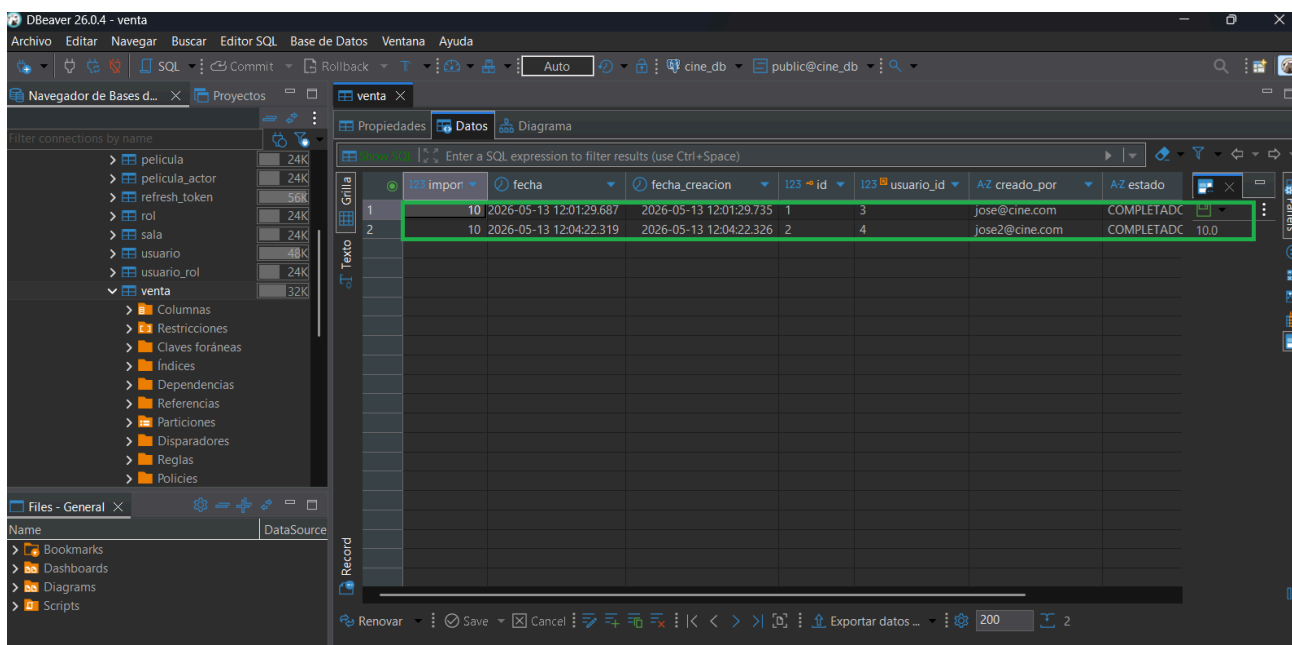
```
1 {
2   "email": "jose@cine.com",
3   "message": "Token refrescado con éxito.",
4   "accessToken": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJqb3NlOGNpbmUyZ2tIiwicm9sZXMiOiJ1c2Vybm9udGVzIiwiaWF0IjoxNjU2fQ.-v2QPJ60ICkCAGhr4htU0truk76xbgFmGUT3hN1344",
5   "refreshToken": "8af913ce-6163-4b34-8928-b5133f8a48e0"
6 }
```


Prueba 8: Auditoría Automática JPA (Prueba de Base de Datos)

Aplicación a usar: DBeaver, PgAdmin, IDE Database Tools, etc.

Por qué lo probamos: Para demostrar un requerimiento Enterprise crítico: saber el "quién" y el "cuándo" de cada registro de venta o entrada en el sistema.

Qué hace y demuestra: Demuestra la integración que hicimos de **Spring Data JPA Auditing**. Al lanzar un **INSERT** en la Base de Datos desde el controlador de ventas, los **@EntityListeners** de Hibernate saltan y ejecutan nuestra clase **AuditorAwareImpl** antes de guardar, insertando las columnas de creación sin ensuciar ni una línea de lógica de negocio en los controladores:



The screenshot shows the DBeaver interface with the 'venta' table selected. The table structure and data are as follows:

	id	fecha	fecha_creacion	id	usuario_id	creado_por	estado
1	10	2026-05-13 12:01:29.687	2026-05-13 12:01:29.735	1	3	jose@cin.com	COMPLETADC
2	10	2026-05-13 12:04:22.319	2026-05-13 12:04:22.326	2	4	jose2@cin.com	COMPLETADC 10.0

NOTA: en nuestro caso hemos descargado y configurado nuestro acceso a la bbdd con **DBeaver Community**